САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 2
Проектирование и технический дизайн API

Выполнил:

Субботин Александр

Группа К3339

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

1 Задача

Тема: Сервис аренды недвижимости. Нужно:

- описать требования (функциональные и нефункциональные),
- выбрать формат API (REST или BFF),
- спроектировать API в OpenAPI с телами запросов/ответов и ошибками.

Стек: Go, Gin, PostgreSQL, OpenAPI/Swagger, микросервисы.

2 Ход работы

2.1 1. Функциональные требования

1. Регистрация и вход.
2. Личный кабинет: мои объекты + мои бронирования.
3. Поиск: `city`, `property_type`, `price`, `rooms`, `max_guests`.
4. Карточка объекта: описание + фото + удобства + правила.
5. Бронирования: создать, подтвердить/отклонить, отменить, завершить.
6. Чаты и сообщения: список чатов, история, отправка сообщений.
7. Избранное и отзывы.

2.2 2. Нефункциональные требования (кратко)

Атрибут	Требование
Безопасность	JWT (Bearer), пароли — хэши (bcrypt/argon2), права на изменения только у владельца.
Валидация	Проверка полей и дат, понятные ответы об ошибках.
Производительность	Пагинация, индексы под поиск (city/type/price), ограничение `limit` до 100.
Поддерживаемость	Версия `/api/v1`, единый формат ошибок, OpenAPI как контракт.

2.3 3. Выбор формата API

Выбран формат: **REST API** (версия `/api/v1`, JSON, JWT).

2.4 4. Общие правила API

Auth: `Authorization: Bearer <token>`

Пагинация: `limit`, `offset`

Ошибка:

- `code` — код ошибки
- `message` — текст

— details — детали

2.5 5. Спецификация эндпоинтов (таблицей)

Ниже — компактный список “ручек” (REST, /api/v1). Для всех защищённых — Authorization: Bearer <token>.

Метод	Путь	Тело запроса	Ответ / ошибки
POST	/auth/register	email, password, full_name, phone?	201 User; 409 email/phone заняты; 400
POST	/auth/login	email, password	200 access_token; 401
GET	/users/me	-	200 User; 401
GET	/properties	Query: city?, property_type?, price_min?, price_max?, rooms?, max_guests?, limit, offset	200 {items, pagination}; 400
POST	/properties	title, description, property_type, price_per_night, city, max_guests, rooms?, amenity_ids?[]	201 Property; 401; 400
GET	/properties/{propertyId}	-	200 {property, photos, amenities}; 404
PATCH	/properties/{propertyId}	title?, description?, price_per_night?, city?, rooms?, max_guests?, status?, amenity_ids?[]	200 Property; 401; 403; 404
DELETE	/properties/{propertyId}	-	204; 401; 403; 404
GET	/users/me/properties	Query: limit, offset	200 {items, pagination}; 401
POST	/bookings	property_id, start_date, end_date, guests_count	201 Booking; 401; 409 даты пересекаются; 422 даты неверные
GET	/bookings	Query: status?, limit, offset	200 {items, pagination}; 401

GET	/users/me/owner/bookings	Query: status?, limit, offset	200 {items, pagination}; 401
GET	/bookings/{bookingId}	-	200 Booking; 401; 403; 404
PATCH	/bookings/{bookingId}	status: {approved,rejected, cancelled,completed}	200 Booking; 401; 403; 404; 422
GET	/chats	Query: limit, offset	200 {items, pagination}; 401
GET	/chats/{chatId}/messages	Query: limit, offset	200 {items, pagination}; 401; 403
POST	/chats/{chatId}/messages	text	201 Message; 401; 403
GET	/favorites	-	200 Property[]; 401
POST	/favorites/{propertyId}	-	201 Favorite; 401; 404; 409 уже в избранном
DELETE	/favorites/{propertyId}	-	204; 401
POST	/reviews	booking_id, rating, text?	201 Review; 401; 403; 409

Примечание: форматы ответов: +- User, Property, Booking, Chat, Message, Favorite, Review — соответствуют моделям из ДЗ1. +- pagination = {limit, offset, total}. +- ошибки возвращаются как {code, message, details?}.

2.5.1 6. Ошибки (примеры)

- POST /auth/register → 409 если email занят.
- POST /bookings → 409 если даты пересекаются; 422 если даты неверные.
- PATCH /properties/{id} → 403 если не владелец.
- GET/POST /chats/{id}/messages → 403 если не участник.
- POST /reviews → 409 если отзыв уже есть или сделка не завершена.

3 Вывод

Спроектирован REST API (Go/Gin) для сервиса аренды недвижимости: заданы требования и подготовлен OpenAPI-контракт с телами запросов/ответов и типовыми ошибками.